

생성형 AI를 활용한 소프트웨어 개발

요약본

전광일 · 최종무

2026

I AI 시대의 소프트웨어 개발

한 줄 요약 생성형 AI(Artificial Intelligence, 인공지능)는 생성을 압축하고 검증의 중요성을 확장함 → 개발자는 코드 작성자에서 설계자·위임자·검증자로 재정의됨

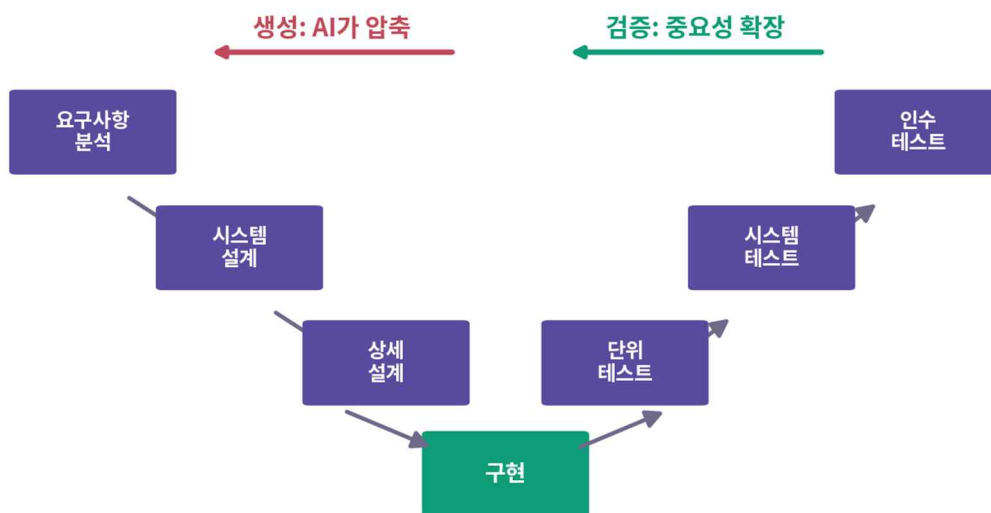
성격	교재 전체의 관통 명제와 판단 프레임(위임 수준·마이크로 사이클) 제시
핵심 개념	관통 명제 · Spec-Scrum-Flow · L1~L4 · 마이크로 사이클
대응 실습	실습 1 (AI 유/무 비교 개발 실험)

1. 패러다임 전환: 무엇이 같고 무엇이 다른가

- 추상화 역사(기계어→고급 언어→프레임워크)의 연장 — 매 전환기의 "프로그래머 소멸" 예측과 달리 실제로 일어난 것은 역할의 재정의
- 결정적 차이: 컴파일러는 결정적(출력을 의심할 필요 없음), 생성형 AI 는 확률적 — 그럴듯하지만 틀린 출력(가짜 함수·경계 오류·요청 밖 기능)을 자신 있게 생성
- 산출물 정확성 보증 책임이 도구 → 사용자로 이동: 이전의 어떤 도구 전환에도 없던 새 조건
- 기본기의 용도 변화: 쓰기 위한 것 → AI 산출물을 읽고 의심하고 판단하기 위한 것
- 설계·검증의 실체는 깊이 생각하는 능력(사색·사고) — 결정 요약·설명 질문·사람끼리 먼저 15 분이 생각을 건너뛰지 못하게 하는 강제 장치
- 개발 방식은 전형적 → AI-powered → AI-driven 의 스펙트럼(표 1-1-1-2) — 프로젝트 고정이 아니라 작업별 선택이며, 오른쪽으로 갈수록 인간 검토가 성패를 좌우

2. 관통 명제: 생성의 압축, 검증의 확장

- V-Model 왼쪽(요구→설계→구현, 만드는 계열)은 AI 로 극적 압축, 오른쪽(확인하는 계열)은 압축되지 않음
- 검증의 본질 = 무엇을 확인할지의 판단 + 결과에 대한 책임 — 타이핑이 아니므로 압축 불가, 미검증 산출물이 쏟아질수록 오히려 부담 증가
- 결론: 개발의 병목은 생성에서 검증으로 이동 — 개인 대응은 마이크로 사이클, 팀 대응은 하이브리드 프로세스와 IX장 운영 원칙



개발의 병목이 '생성'에서 '검증'으로 이동한다

〈그림 1-4〉 V-Model의 재해석: 생성의 압축과 검증의 확장

3. 하이브리드 프로세스(Spec-Scrum-Flow)

구분	순수 폭포수	애자일(Scrum 등)	하이브리드(본 교재)
방향 결정	초기에 완전 확정	반복 속 점진 형성	선행 4단계에서 핵심만 결정
선행 4단계	충실하나 시간 소요 큼	소홀해지기 쉬움	문서 중심 1회 수행
검증 주기	후반 집중(오류 누적)	반복마다	2주 스프린트마다 강제
AI 위임 기준	문서는 있으나 경직	명세 부실 시 모호	수용 기준 붙은 이슈 단위

- 구조: 선행 4 단계(아이디어→기획→요구→시스템 설계, 문서 중심 1 회) + 반복 5 단계(상세 설계→구현→테스트→배포→운영, 2 주 스프린트)
- 두 세계의 다리 = Product Backlog: 선행 산출물이 이슈로 분해되어 쌓이고, 운영의 발견이 되돌아오는 유일한 접점
- 짧은 반복 = 미검증 AI 산출물 축적의 구조적 차단 / 잘 쓰인 이슈 = 그 자체로 AI 위임 단위
- 이름의 의미: Water 가 아니라 Spec(쪽 상한·결정 요약의 얇은 스펙 — "스펙이 새로운 코드다"), Fall 이 아니라 Flow(게이트를 통과한 증분의 자동 배포) — AI 리스크 대응을 위해 설계된 결합

4. AI 코딩 도구의 세 갈래

갈래	주 용도 (강점)	주의점
챗 기반	설계 논의·개념 학습·문서 초안 등 탐색적 작업	코드베이스와 분리 — 맥락 요약의 품질이 답변의 품질
IDE(Integrated Development Environment, 통합 개발 환경) 인라인	작성 중 코드의 실시간 보완·국소 수정	‘받아들이기’의 유혹 — 이해 못 한 코드의 유입
에이전틱	완결된 작업 단위의 통제 위임(멀티파일 구현)	자율성만큼 커지는 검증 부담 — 완료 조건·리뷰 필수

- 선택 기준은 작업의 성격 — 한 이슈 안에서도 갈래를 오감, 판단이 서지 않으면 챗으로 시작(완료 조건이 뚜렷해지면 위임으로 전환)

5. 위임 수준 모델(L1~L4)

수준	정의	적용 단계	핵심 주의
L1 조언자	의견·대안·리서치만, 작성·결정은 사람	아이디어·기획	AI가 먼저 쓰면 앵커링 — 문제의식은 위임 불가 자산
L2 초안자	AI 초안 + 사람 검토·확정	요구·설계	초안의 품질 ≠ 확정본의 품질 — 차이는 검토가 만듦
L3 위임	완료 조건 정의 후 통제 위임, AI 자가 검증	구현·테스트·유지보수	성패는 위임 전 완료 조건에서 결정
L4 자율	게이트 갖춘 파이프라인의 자동 실행	배포·CI/CD(Continuous Integration/Continuous Delivery)	자율은 신뢰의 결과 — 게이트 없는 자율은 금지

위임 수준 판단 질문 (표 1-6)

점검 질문	‘예’라면	‘아니오’라면
-------	-------	---------

이 작업의 산출물이 곧 판단·결정인가?	L1 — 조언만 구한다	다음 질문으로
완료 조건을 검증 가능한 형태로 쓸 수 있는가?	L3 위임 가능	L2 — 초안을 받아 다듬는다
실패를 자동 감지·차단하는 게이트가 있는가?	L4 자율화 검토 가능	L3까지만 — 게이트부터 구축

- 수준이 올라갈수록 성패의 두 축 = 사전 명세(스펙·완료 조건) + 사후 검증(테스트·게이트)의 품질
- 학습자 추가 축: 그 영역을 내가 아는가 — 처음 보는 영역은 한 단계 낮춰 직접 쓰고, 반복 작업은 게이트를 만들어 넘김(표 1-7). 어느 칸에서도 위임하지 않는 것은 검증

6. 마이크로 사이클과 7. 다섯 가지 오해

- 모든 AI 협업의 기본 루프: 정의(무엇을·완료 판단 기준) → 생성 → 검증(테스트·실행·리뷰) → 수정(검증 정보를 담은 재지시) — 검증을 통과한 산출물만 다음 작업의 입력
- 원샷 사고 = 대표적 안티패턴: 한 방을 노리다 실패하면 "AI 가 못 한다"로 결론 — 첫 산출물은 완성물이 아니라 AI 의 이해를 보여 주는 진단 자료

다섯 가지 오해의 교정

오해	교정
"기본기는 덜 중요해졌다"	반대 — 검증은 기본기 위에서만 가능, 용도가 쓰기→읽고 의심 하기로 변화
"좋은 프롬프트 한 번이면 된다"	협업은 원샷이 아니라 루프 — 검증이 만든 정보가 다음 지시를 정확하게
"컴파일되고 실행되면 맞는 코드다"	‘돌아간다’와 ‘옳다’는 다른 말 — AI 오류의 무서움은 그럴듯함
"AI 사용은 부정행위다"	기준은 사용 여부가 아니라 공개와 검증 — 설명 못 하는 산출물의 제출만이 부정행위
"도구를 많이 알수록 잘 쓴다"	도구는 바뀔 — 오래 남는 것은 판단 프레임(갈래 구분·위임 수준)

II LLM 의 원리와 한계, 그리고 소통법

한 줄 요지 토큰·다음 토큰 예측·컨텍스트 윈도우라는 원리에서 세 한계(환각·컷오프·비결정성)가 필연적으로 나오며, 대응은 4요소 프롬프트·컨텍스트 엔지니어링·4가지 횡단 습관

성격	AI 협업의 원리적 기초
핵심 산출물	팀 프롬프트 표준 초안
대응 실습	실습 2 (환각 유도·전략 비교)

1. 개발자에게 필요한 만큼의 원리

- 토큰: LLM(Large Language Model, 대형 언어 모델)이 텍스트를 다루는 조각 단위 — 요금(토큰당 과금)·입력 길이 제한·응답 속도가 모두 이 단위 위에서 결정
- 다음 토큰 예측: 확률분포 계산 → 하나 선택 → 덧붙임의 반복 — LLM 은 사실의 DB(Database, 데이터베이스)가 아니라 패턴의 예측기(가장 그럴듯한 다음 말 ≠ 참인 말), 확률 개입으로 같은 질문에 다른 답 가능
- 컨텍스트 윈도우: 한 번에 볼 수 있는 토큰 총량 — 창 밖은 존재하지 않는 것과 같고, 대화가 길어지면 앞부터 밀려나 초반 규칙을 ‘잊은 듯’ 행동. 무엇을 넣고 뺄지의 선별이 품질

2. 세 가지 한계와 하나의 원칙

한계	본질·원인	대응
환각	버그가 아니라 구조 — 예측기는 ‘모른다’를 출력하도록 만들어지지 않음. 근거 있는 답과 없는 창작이 같은 유창함(가짜 함수·통계·인용)	출력의 기본값을 ‘검증 전 초안’으로 — 유창함에 신뢰를 선지불하지 않음
지식 컷오프	학습 시점 이후를 모르면서 옛 지식으로 자신 있게 답변	최신 정보는 웹 검색 결합 도구, 버전이 중요한 코드는 공식 문서 대조
비결정성	확률적 선택 — 같은 프롬프트, 다른 결과	발산에는 자산으로 활용, 재현 필요 작업은 형식 강한 고정 + 한 번의 출력에 의존하지 않기



〈그림 2-4〉 프롬프트의 4요소 구조 (러닝 케이스 예)

3. 프롬프트 기본기

- 4 요소 구조: 역할(어떤 관점)· 목표(무엇을)· 제약(지킬 조건·금지)· 출력 형식(어떤 모양) — 자주 빠지는 것이 제약·형식이고 그 결과가 일반론 답변
- few-shot: 원하는 출력의 예시 1~2 개 — 형식 작업에서 설명보다 강력, 단 예시가 곧 틀이므로 나쁜 예시는 나쁜 출력을 복제

- 반복적 개선: 부족을 구체적으로 지적하며 좁히기("오류 케이스가 없다. 빈 검색어와 0 건을 추가하라") — 개선 지시의 품질이 곧 검증의 품질

프롬프트 안티패턴과 교정 (표 2-2)

안티패턴	증상	교정
모호한 지시	‘좋게’, ‘빠르게’ 같은 형용사만	판단 가능한 기준으로 정량화(‘1초 이내’)
과적재	한 프롬프트에 서로 다른 작업 여러 개	작업을 분해해 한 번에 하나씩
유도 질문	‘이 설계 괜찮지?’ — 동의 요구	‘이 설계가 실패할 이유 3가지’로 반전
맥락 없는 조각	코드 일부만 던지고 ‘고쳐 줘’	목적·오류 증상·관련 코드를 함께 제공

4. 컨텍스트 엔지니어링

- 프롬프트가 ‘어떻게 말할까’라면 컨텍스트는 ‘무엇을 보여줄까’ — 후자가 대개 더 결정적(답이 이상한 원인의 절반 이상은 재료)
- 선별 3 질문: 이 판단에 꼭 필요한가 / 없으면 잘못 추측할 정보인가 / 요약으로 충분한가 — 전부 주는 것은 능사가 아님
- 규약 파일: 스택·구조·컨벤션·금칙을 저장소에 — 도구가 자동으로 읽으며, AI 가 반복해 틀리면 즉시 한 줄 추가(살아 있는 문서)
- 세션 위생: 앞뒤 안 맞는 답·폐기 시안의 재등장 = 오염 신호 — 긴 설명이 아니라 초기화: ‘지금까지의 결정 요약’을 받아 새 세션으로

5. 전 단계를 관통하는 4 가지 습관

습관	내용
컨텍스트 관리	프로젝트의 구조·규약·용어를 AI가 항상 알 수 있는 상태로 — 컨텍스트의 품질이 곧 출력의 품질
프롬프트·의사결정 기록	무엇을·왜·어떻게 시켰는지를 작업과 동시에 기록 — 재현성·팀 공유·과정 중심 평가의 근거
보안·라이선스 상시 점검	산출물 불신이 기본값 — 비밀키 유입·가져온 코드의 라이선스 확인 절차의 습관화
비용·시간 관리	토큰·구독 비용 + 검증 시간 — ‘생성 1분, 검증 1시간’의 반복은 위임 수준 재조정 신호

III 아이디어 도출 단계

한 줄 요지 목표는 착상이 아니라 만들 가치 있는 문제의 발견·정의 — AI는 폭을 넓히는 조언자(L1), 판단과 문장은 사람. 이 단계 최대의 위험은 앵커링

적정 위임 수준	L1 조언자
핵심 산출물	문제 정의서 (2쪽 상한)
대응 실습	실습 3 (주제 선정)

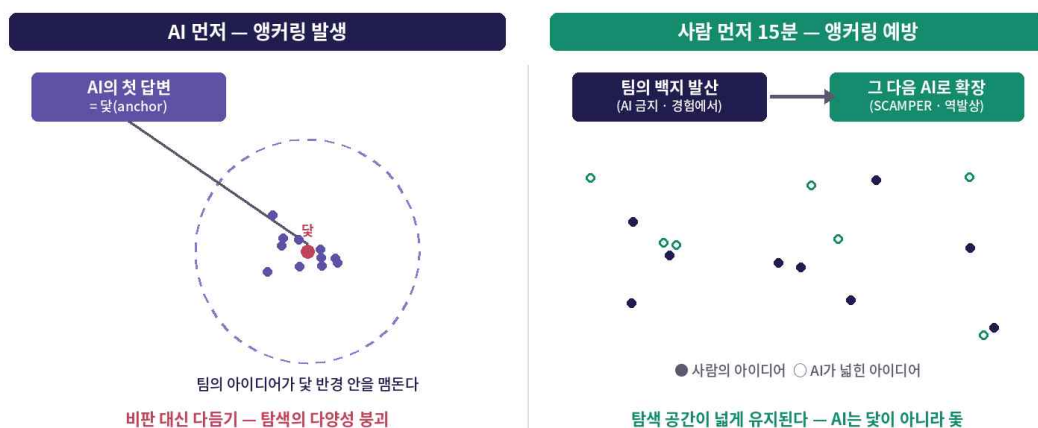
1. 산출물의 해부: 문제 정의서

요소	내용
① 문제 진술	누가, 어떤 상황에서, 무엇 때문에 불편한가
② 대상 사용자	그 불편을 가장 자주·아프게 겪는 사람
③ 기존 대안과 한계	지금 어떻게 해결하고 있으며 왜 불충분한가
④ 해법 가설	무엇을 만들어 해결하려 하는가 (한 문단)
⑤ 결정 요약	검토했으나 배제한 아이디어와 배제 이유 — 문서 품질의 관건

- 결정 요약이 있어야 다음 단계에서 "그 아이디어는 왜 안 했지?"의 회귀가 방지됨 — 하지 않기로 한 것이 하기로 한 것만큼 중요

2. 왜 L1 인가: 앵커링

- 앵커링: 먼저 접한 정보가 닳아 되어 판단이 그 주변을 벗어나지 못하는 인지 편향 — 임의적임을 알면서도 기준점 삼아 ‘조정’하며, 조정은 대개 불충분
- AI 가 위험한 닳은 이유 3: 빠르고 유창해 백지의 불안을 즉시 해소 / AI 의 답 = 학습 데이터의 평균적 답(탐색이 혼한 지점으로 수렴) / 틀이 오면 에너지가 대안 탐색이 아니라 다듬기로 유출
- 처방은 도구 폐기가 아니라 순서: 사람끼리 먼저 15 분 — 백지 발산 후 AI 를 열면 같은 AI 가 닳아 아니라 뚫이 됨



〈그림 3-1〉 앵커링: 먼저 도착한 답이 사고의 반경을 정한다

역할 분담 (표 3-1)

개발자(팀)가 하는 일	AI가 하는 일
문제·기회의 정의, 브레인스토밍 주도	트렌드·경쟁 서비스 리서치 (웹 검색 결합)

아이디어의 시장성·실행 가능성 판단	아이디어 확장·변형·결합 제안
이해관계자 의견 수렴	유사 사례 벤치마킹과 시사점 도출
최종 주제 선택과 문제 정의서 작성	가상 사용자 반응 시뮬레이션

3. AI 활용 기법

- 확산·수렴 분리: 확산의 AI 는 양의 기계(SCAMPER(Substitute·Combine·Adapt·Modify·Put to other uses·Eliminate·Reverse) 변형, "반드시 실패하게 만들려면?" 역발상으로 숨은 전제 노출) / 수렴의 AI 는 기준의 거울 — 순위 결정은 팀
- 벤치마킹: 웹 검색 결합 필수(킵오프) — 산출물은 기능 비교표+차별점 후보, 전 항목에 출처
- 가상 사용자 시뮬레이션: 페르소나를 구체화할수록 반응의 밀도 상승, 서로 다른 페르소나 서너 명으로 관점 순회 — 용도는 실제 인터뷰의 대체가 아니라 예행연습(질문지 다듬기·예상 반론), 실제 인터뷰는 생략 불가

1 차 스크리닝 4 기준

기준	묻는 것
빈도·강도	자주, 아프게 겪는 문제인가
기술적 실현 가능성	우리 팀의 기술로 만들 수 있는가
기존 대안 대비 차별성	이미 있는 해법과 무엇이 다른가
학기 내 완성 가능성	스프린트 3회 안에 핵심을 완성할 수 있는가

- 표의 칸에는 판단이 아니라 근거 사실만 적게 하고, 최종 점수와 선택은 팀 토론 — 이 토론의 기록이 결정 요약의 원천
- 프롬프트 요령: 제약 없는 요청("좋은 앱 아이디어")은 일반론을 부름 — 역할·팀 조건·발산 지시·형식을 갖추고, 수렴 요청의 마지막 문장 "순위는 매기지 마라"가 결정권을 팀에 남김

이 단계의 검증 체크리스트

- 문제 정의서의 모든 수치에 확인 가능한 출처가 있는가 (없으면 ‘팀 추정’ 표기)
- 최종 후보에 ‘실패할 이유’를 물었고 그 답을 결정 요약에 반영했는가
- 동일·유사 서비스의 존재를 웹 검색으로 확인했는가
- AI 를 열기 전에 사람끼리 먼저 브레인스토밍했는가 (앵커링 방지)
- 배제한 아이디어와 이유가 결정 요약에 기록되어 있는가

전형적 실수와 검증

전형적 실수	검증
출처 없는 통계 생성 — 숫자가 붙으면 사실로 취급됨	모든 수치에 출처 요구·원자료 확인, 없으면 ‘팀 추정’ 명시
아부 편향 — 모든 아이디어가 ‘훌륭한 잠재력’을 갖게 됨	‘실패할 이유 5가지’ 역방향 평가+ 발산·평가 세션 분리
이미 존재하는 서비스의 재발명	최종 후보 2~3개는 웹 검색으로 확인 — 존재하면 벤치마킹 대상(모르는 것만이 손해)

IV 기획 단계

한 줄 요지 풀 만한 문제(III장)를 ‘우리가·이 자원으로·이 기간에 풀 수 있는가’로 전환 — 결정은 L1, 전개는 L2. AI는 낙관에 찬물을 끼얹는 회의적 검토자로 쓸 때 가장 유용

적정 위임 수준	L1 조언자 ~ L2 초안자
핵심 산출물	프로젝트 개요서·로드맵 (4쪽 상한)
대응 실습	실습 4 (개요서 작성)

1. 산출물의 해부: 프로젝트 개요서

요소	내용
① 목표와 성공 기준	학기말에 무엇이 어떤 상태면 성공인가 — 측정 가능하게
② 범위	MoSCoW(Must·Should·Could·Won't)로 정리된 기능 우선순위 — 특히 Won't(하지 않을 일) 목록
③ 로드맵	스프린트 단위의 목표 (날짜 단위 세부 일정이 아님)
④ 리스크와 대응	프리모템에서 도출된 상위 리스크와 대응 계획
⑤ 결정 요약	검토했으나 배제한 방향과 그 이유

- 기획 = 판단(L1)과 문서 작업(L2)의 혼합 단계 — 목표·범위·우선순위의 결정은 L1, 정형 양식 전개는 L2: 결정은 L1, 전개는 L2 가 장 전체의 운영 원리

역할 분담 (표 4-1)

개발자(팀)가 하는 일	AI가 하는 일
목표·성공 기준의 확정	린 캔버스·개요서 초안 전개 (L2)
범위와 우선순위 결정 (MoSCoW)	유사 프로젝트 사례·시장 조사 요약 (L1)
리스크의 심각도 판단과 대응 선택	프리모템 실패 시나리오 도출 (L1~L2)
일정·자원의 최종 약속	WBS(Work Breakdown Structure, 작업 분해 구조) 초안과 추정 근거 제시 (L2)

Must 반드시	Should 가능하면	Could 여유 있으면	Won't 이번엔 안 함
건물 검색·조회 현장 브리핑 접근성·데이터 기반	사전 학습 모드 LLM 문안 생성 포인팅·터치 탐색	즐거찾기 반복 청취 개인화	실시간 내비게이션 스마트워치 지역 확장

Won't 목록이 곧 결정 요약이다 — ‘안 하기로 한 것’을 명시해야 범위가 지켜진다

<그림 4-3> 러닝 케이스의 MoSCoW: Won't가 범위를 지킨다

2. AI 활용 기법

- 린 캔버스: 문제 정의서를 컨텍스트로 초안 전개(L2) — 더 가치 있는 용법은 채워진 캔버스의 ‘비어 있거나 근거 약한 칸’ 지적(상습 약점: 채널·핵심 지표)

- 이해관계자 시뮬레이션: 회의적 지도교수·예산 심사자 역할로 캔버스를 공격시키기 — 발표 전 예행연습
- 일정·리소스 추정: AI 추정은 낙관적이고 근거가 숨어 있음 → 숫자가 아니라 가정을 요구("어떤 가정 위에서 3 일인가 — 숙련도·재사용·검증 포함 여부") / 모든 추정에 검증·리뷰 시간 명시 + 전체 20~30% 버퍼
- 프리모템: '이미 실패했다' 가정 후 원인 역추적 — 미래 위험에는 낙관하고 일어난 실패에는 구체적으로 답하는 심리적 비대칭 활용. 아부 편향을 우회해 AI 가 신랄해지는 역할 반전 — 상위 3~5 건을 대응 계획으로 바꾸는 것은 팀의 몫
- MoSCoW: Must(없으면 서비스가 아님)/Should/Could/Won't — 범위 폭발은 학기 프로젝트의 제 1 사인(死因). Won't 목록이 곧 결정 요약: 안 한다고 적지 않은 기능은 스프린트 중반에 반드시 되돌아옴
- 로드맵: 단위는 날짜가 아니라 스프린트 목표 — 세부 일정은 각 스프린트 Planning 에서 그때의 정보로

이 단계의 검증 체크리스트

- 성공 기준이 15 주차에 측정 가능한 문장으로 적혀 있는가
- 모든 일정 추정에 가정과 검증 시간이 명시되어 있는가 — 버퍼(20~30%)가 있는가
- 프리모템 상위 3 개에 구체적 대응 계획이 붙어 있는가
- Won't 목록이 존재하며 배제 이유가 결정 요약에 있는가
- 로드맵이 날짜가 아니라 스프린트 목표 단위인가

전형적 실수와 검증

전형적 실수	검증
근거 없는 낙관 추정 — 이상적 조건(숙련·무방해·검증 생략)을 암묵 전제	가정 명시 + 반대 시나리오("2배가 되면 어떤 가정이 깨진 것인가") 대조 — 일정은 약속이 아니라 가설, 실속도는 Sprint 1 실측으로만
리스크의 과소평가와 일반론("일정 지연 가능성"류)	프리모템 형식으로 구체적 사건 요구 + "우리의 어떤 사실이 이를 가능하게 하는가"로 필터링
범위의 슬금슬금 확장(scope creep) 방조	발산 후 성공 기준·용량 재제시로 수렴 강제 — 버린 기능은 Won't/다음 학기 백로그로 명시적 기록

V 요구사항 분석과 정의 단계

한 줄 요지 스펙이 새로운 코드 — 사람이 직접 만드는 것은 코드가 아니라 코드의 기준이 되는 명세이고, 그 품질이 AI 산출물의 상한. 원천은 사람, 정형화는 AI, 확정 책임은 사람

적정 위임 수준	L2 초안자
핵심 산출물	SRS(Software Requirements Specification, 소프트웨어 요구사항 명세서)·수용 기준 (10쪽 상한)
대응 실습	실습 5 (인터뷰→SRS)

1. 산출물의 해부: SRS

구성	내용
① 개요와 범위	개요서의 MoSCoW를 계승
② 기능 요구	Must·Should 기능별 유저 스토리 + 수용 기준 (EARS(Easy Approach to Requirements Syntax) 문형)
③ 비기능 요구	성능·보안·개인정보·운영
④ 용어집	팀 용어의 정의
⑤ 결정 요약	검토했으나 배제한 요구와 이유

- 모든 요구에 R-번호 식별자 — VII장 이슈, XII장 테스트로 이어지는 요구 추적성의 실
- 스토리 = ‘무엇을 원하는가’의 대화, 수용 기준 = ‘무엇이 되면 끝인가’의 계약 — 한 번 잘 쓴 수용 기준이 세 번 일함(이슈 본문·위임 완료 조건·테스트 원본), 좋은 스토리의 기준은 INVEST(Independent·Negotiable·Valuable·Estimable·Small·Testable)

2. AI 활용 기법

- 소크라테스식 질의: 바람("잘 됐으면") → 검증 가능한 요구로 내려가는 질문 사다리 — "검증 가능하게 만들기 위해 물어야 할 질문 5 개" 요청으로 인터뷰 준비, 페르소나로 예행연습
- 3 대 결함(누락·모순·모호) 탐지: 자기 문서의 결함에 관대한 사람 대신 AI 가 대량 탐지 — 단 AI 가 찾는 것은 후보, 판정과 수정은 사람(의도된 설계일 수 있음)
- 비기능 요구 = 물어야 나오는 빙산의 아랫부분 — 범주별(성능·보안·개인정보·접근성·운영) 체크리스트 요청이 인터뷰 보조 자료
- SRS 워크플로우: 입력(정의서·개요서·인터뷰) → AI 초안(L2, 근거 밖 내용은 [제안] 분리) → 3 관문(결함 점검·비기능 보강·결정 요약) → 확정 → 설계(VI)와 테스트(XII)의 공통 기준으로

EARS: 요구를 다섯 문형으로 정형화한다

보편형	시스템은 항상 ~해야 한다 "시스템은 게시 상태의 건물만 검색과 브리핑에 노출해야 한다"
사건 구동형	<사건>이 발생하면 ~해야 한다 "'브리핑 시작'이 실행되면, 반경 50m 밴드부터 시계방향으로 안내해야 한다"
상태 구동형	<상태>인 동안 ~해야 한다 "사전 학습 모드인 동안, 매 브리핑 시작 시 '가상 위치 기준'임을 알려야 한다"
선택형	<조건>인 경우 ~해야 한다 "밴드 내 건물이 5개를 초과하는 경우에만, '외 N개'로 요약해야 한다"
원치 않는 동작	<오류>가 발생하면 ~해야 한다 "검색 결과가 0건이면, 안내 문구를 표시해야 한다"

EARS 다섯 문형

문형	형태
보편형	(항상) 시스템은 ~해야 한다
사건 구동형	~가 발생하면, 시스템은 ~해야 한다
상태 구동형	~인 동안, 시스템은 ~해야 한다
선택형	~인 경우에만, 시스템은 ~해야 한다
원치 않는 동작	오류·실패가 발생하면, 시스템은 ~해야 한다

- 변환 요청 시 [분류 불가] 표시 강제 — 그 목록이 보물: 문형에 담기지 않는 요구는 대개 아직 바람이며 사다리로 되돌릴 대상

이 단계의 검증 체크리스트

- 모든 요구에 식별자(R-번호)가 있고 EARS 문형에 담겨 있는가
- 모든 수용 기준이 ‘시험 가능’ 판정을 통과했는가
- 모순·누락·모호 탐지를 별도 세션에서 수행하고 판정 기록을 남겼는가
- 비기능 요구가 범주별로 존재하는가
- [제안]으로 분리된 AI 창작분이 본문에 섞이지 않았는가 — 결정 요약이 있는가

전형적 실수와 검증

전형적 실수	검증
모순·검증 불가능한 수용 기준 — AI는 전역 일관성 미보장	결함 탐지를 별도 세션에서 + 전 기준에 시험 가능성 판정("확인 테스트를 한 문장으로 쓸 수 있는가")
해법을 요구로 기술("○○ 엔진을 사용해야 한다")	what/how 분리 질문 — how는 설계 메모로 VI장에 이관(요구 단계의 해법 확정은 설계 공간을 이유 없이 축소)
문서 인플레이션 — 초안이 공짜라 SRS가 저절로 비대	10쪽 상한 + "이 문단을 지우면 사라지는 결정이 있는가" — 없으면 삭제

VI 시스템 설계 단계

한 줄 요지 설계에 정답은 없음 — 팀의 조건 아래의 더 나은 트레이드오프뿐이고, 트레이드오프는 비교가 있을 때만 보임. 제1 규칙: 대안은 항상 셋 이상

적정 위임 수준	L2 초안자
핵심 산출물	시스템 설계 문서(8쪽 상한)·ADR(Architecture Decision Record, 아키텍처 결정 기록)
대응 실습	실습 6 (3안·ADR·레드팀)

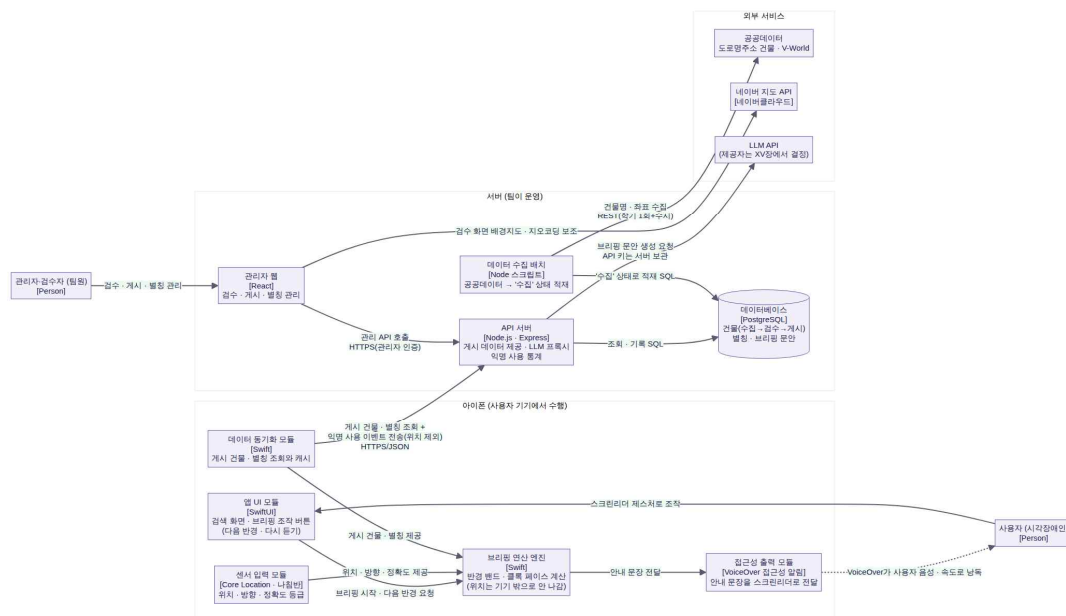
1. 산출물의 해부: 설계 문서와 ADR

산출물	역할·구성
시스템 설계 문서 (8쪽)	현재의 구조 — 아키텍처 개요(C4 컨테이너 구조도)·기술 스택과 이유·주요 컴포넌트와 책임·데이터 전략 개요·결정 요약
ADR (결정당 1장)	왜 이 구조가 되었는가의 역사 — 제목·상태·맥락·결정·근거·배제 대안·결과의 7항목

- 상태 관리 규율: 전제가 깨지면 원본 수정이 아니라 새 ADR 추가 + 관계 표기 — 결정이 결정을 밀어내면 superseded, 본문 전제의 붕괴면 보완. 설계 변경은 실패가 아니라 학습의 증거(IX장 원칙 4)

2. AI 활용 기법

- 3 안의 규율: 축을 지정한 발산(가장 단순/균형/확장성 극대화) + 추천 금지 — 배제안은 버려지지 않고 ADR의 ‘배제 대안’ 칸에 기록되어 훗날의 물음에 답하는 재산이 됨
- 스택 비교: 비교의 프레임(기준·후보·표)은 AI, 사실의 확인(버전·유지보수·라이선스)은 웹 검색·공식 문서 — 비교 기준에 팀의 현재 역량 필수: 처음 배우는 스택은 그 자체가 리스크
- ADR 전개: 팀 토론 기록을 주고 7 항목 배치를 맡기는 전형적 L2 — 근거 없는 항목은 [토론 필요]로 남겨 다음 회의의 안건으로
- 다이어그램: Mermaid 텍스트 관리(버전 관리·AI 수정·diff 리뷰), C4 컨텍스트+컨테이너 2 단계면 충분 — 검토 규칙: 라벨 없는 화살표는 설계가 아님(무엇이 오가는지 말할 수 없는 연결은 미결정)



〈그림 6-3〉 러닝 케이스의 컨테이너 다이어그램 (C4 2단계)

AI 레드팀 설계 리뷰

공격 관점	무엇을 보는가
단일 장애점	하나가 죽으면 전체가 죽는 지점
부하 병목	피크 트래픽에서 먼저 무너지는 곳
보안 악용	인증 없는 노출, 업로드·입력의 악용 경로
운영 사각지대	실패가 일어나도 아무도 모르는 곳

- 발견은 ‘즉시 해결’이 아니라 ‘결정’의 대상 — 지금 대응 / 백로그 / 감수의 3 분류로 기록: 전부 해결하려 들면 설계 단계가 끝나지 않음

이 단계의 검증 체크리스트

- 대안이 3 안 이상 존재하고 채택 근거·배제 이유가 ADR 에 있는가
- 모든 구조 요소가 YAGNI(You Aren't Gonna Need It) 질문을 통과했는가
- 스택의 현재 상태(버전·라이선스)를 웹 검색으로 확인했는가 — 팀 역량이 비교 기준에 있는가
- 다이어그램의 모든 화살표에 라벨이 있는가 — 문서와 일치하는가
- 레드팀 발견이 지금 대응/백로그/감수로 분류·기록되었는가

전형적 실수와 검증

전형적 실수	검증
과잉 설계 — 학기 프로젝트에 마이크로서비스·메시지 큐·다중 캐시 제안	전 요소에 YAGNI 질문("없으면 Must 중 무엇이 불가능한가") — 답이 없으면 제거, 더 단순한 안과의 비교가 과잉을 노출
유행 기술의 무비판 추천 — 훈련 데이터 시점의 인기, 팀 역량과 무관	현재 상태는 웹 검색 확인 + "이 스택으로 첫 기능을 Sprint 1 에 끝낼 수 있는가"를 최종 관문으로
다이어그램과 문서의 불일치 — 둘이 다른 시스템을 말함	변경은 ADR → 다이어그램 → 문서의 단일 경로로만 + 불일치 탐지를 AI에게 주기적 위임

VII Scrum 프레임워크와 하이브리드 프로세스

한 줄 요지 계획의 세계에서 경험의 세계로 — 2주 리듬은 미검증 AI 산출물 축적의 구조적 차단 장치이며, 스프린트는 팀 차원의 마이크로 사이클

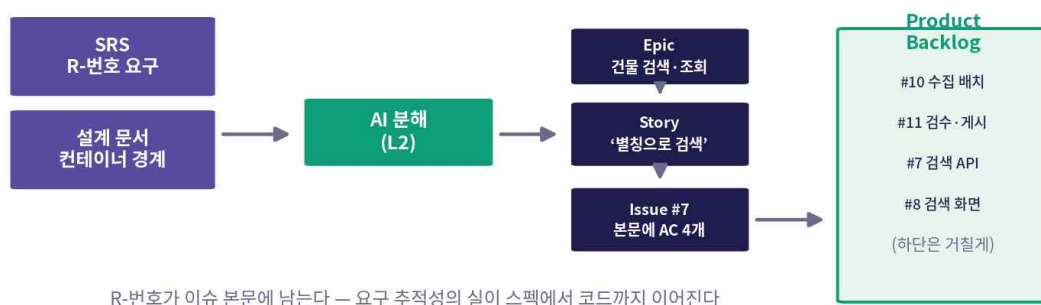
성격	3부의 시작 — 반복 단계의 기반
핵심 산출물	Product Backlog (수용 기준 붙은 이슈)
대응 실습	실습 7 (백로그·Sprint 0 계획)

1. Scrum 의 뼈대: 3-5-3

구분	구성	요점
역할 3	PO(Product Owner, 제품 책임자) · SM(Scrum Master, 스크럼 마스터) · Developers	PO는 가치·우선순위(백로그 순서는 PO의 결정), SM은 ‘Scrum다움’과 장애물 제거, Developers는 증분을 만드는 전원
이벤트 5	Sprint(2주 고정) + Planning · Daily(15분) · Review · Retro	회의가 아니라 검사와 적응의 장치 — 보고를 위한 자리가 되는 순간 형식만 남음
산출물 3	Product Backlog · Sprint Backlog · Increment	Increment에는 완료의 정의(DoD(Definition of Done, 완료의 정의))가 따라붙음 — 실체는 VIII장 파이프라인으로 구현

학생 팀을 위한 적용 규칙 (표 7-1)

항목	적용 규칙
역할 로테이션	PO·SM을 스프린트마다 교대 — 전원이 우선순위 결정과 프로세스 운영을 경험
스프린트 길이	2주 고정, 늘리지도 줄이지도 않음 — 리듬의 예측 가능성이 곧 훈련
AI의 지위	네 번째 팀원이 아니라 모든 역할의 증폭기 — 결정과 책임은 항상 사람
회의 상한	Planning 2시간 · Daily 15분 · Review·Retro 각 1시간 — 넘치면 SM이 끊음



〈그림 7-3〉 백로그 전환: SRS와 설계가 에픽·스토리·이슈로 분해된다

2. 백로그 전환: 스펙에서 이슈로 (L2)

- 분해 3 층: 에픽(큰 기능 묶음) → 유저 스토리(INVEST 단위) → 이슈(수용 기준이 본문에 붙은 실행 단위)
- 자연스러운 경계는 이미 있음 — SRS 의 Must 기능이 에픽 후보, 설계의 컨테이너 경계가 기술 에픽 후보
- AI 의 역할은 전형적 L2: 분해 초안 전개 — 단 R-번호를 이슈 본문에 보존시켜 추적성의 실이 끊기지 않게(코드와 요구의 양방향 추적)

- DEEP(Detailed appropriately·Emergent·Estimated·Prioritized) 원칙: 다음 스프린트 후보만 수용 기준까지 완비된 Ready 로, 아래로 갈수록 거칠게 — 전부 미리 상세화는 폭포수적 낭비(아래쪽은 스프린트가 다가올 때 그때의 지식으로 정제)

3. 스프린트 한 바퀴 (2 주의 리듬)

이벤트	시점	핵심 활동
Planning	Day 1	두 질문 — 이번 스프린트의 목표는(왜) / 어느 항목을 어떻게 나눌 것인가(무엇·어떻게)
개발 + Daily	Day 2~9	이슈→브랜치→PR(Pull Request, 병합 요청) 사이클 + 매일 15분, 보드 앞에서 어제·오늘·블로커를 서로에게 공유
Review	Day 10 오전	동작하는 증분 시연 — 성공 기준은 발표의 매끄러움이 아니라 백로그에 유입된 피드백 수
Retrospective	Day 10 오후	개선 실험 한 가지 결정 + 고정 안건 'AI 활용 회고'(무엇을 시켰고 무엇이 실패했고 어떻게 검증했나 — 백서의 원문)

이 단계의 검증 체크리스트

- 백로그의 모든 이슈에 R-번호와 수용 기준이 있는가 — 출처 없는 항목이 없는가
- 다음 스프린트 후보가 전부 Ready 판정을 통과했는가 — 아래쪽까지 과잉 상세화하지 않았는가
- 이벤트가 시간 상한을 지키는가 — Daily 가 보고회로 변질되지 않았는가
- Review 마다 백로그에 피드백이 실제로 유입되는가
- 회고의 고정 안건(AI 활용 회고)이 기록으로 남는가

전형적 실수와 검증

전형적 실수	검증
형식주의 — Daily가 보고회, Review가 발표회로 변질	검사·적응이 사라지면 회의만 늘린 폭포수 — Daily는 보드 보며 서서 서로에게, Review는 피드백 유입 수로 평가
AI 스토리 공장 — 몇 초의 50개를 검토 없이 투입, 백로그가 쓰레기장화	진입 관문: R-번호 추적성 + PO 우선순위 판정 — 출처 없는 스토리는 아이디어 보관소로
스프린트 중 목표 변경("이것도 급해요")	새 요구는 백로그 → 다음 Planning — 예외는 운영 핫픽스뿐이며 그것도 SM이 기록

VIII GitHub 협업 환경: Repo · Projects · Actions

한 줄 요지 약속은 바쁘면 깨진다 — 규칙을 도구의 구조로 옮겨 VII장의 개념 하나하나를 GitHub의 실물로 전환(스토리→Issue, 백로그→보드, DoD→파이프라인)

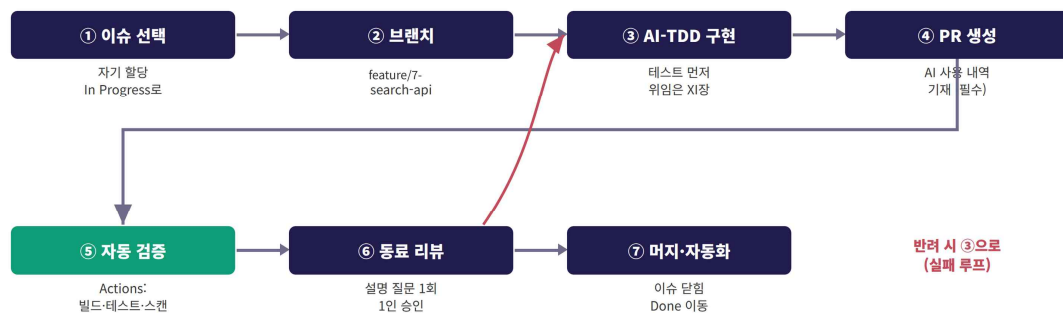
성격	Sprint 0의 실물 셋업
핵심 산출물	팀 저장소·보드·파이프라인
대응 실습	실습 8 (Sprint 0 완성)

1. 왜 GitHub 3 종인가

- 저장소·이슈·보드·자동화가 한곳 — 추적성의 실(R-번호→이슈→PR→커밋)이 도구 경계에서 끊기지 않음
- 모든 활동이 텍스트로 남아 AI의 컨텍스트가 됨 — 이슈 본문·리뷰 코멘트·커밋 메시지가 곧 ‘창에 넣을 재료’
- 에이전틱 도구들의 표준 무대 — 이슈를 받아 PR을 만드는 워크플로우(XI장)가 이 위에서 동작

2. 저장소·이슈·브랜치

- 저장소 뿌리 3 중: README(무엇을·어떻게 실행) / 규약 파일(AI 자동 참조) / docs/(설계 문서·ADR 체인) — 코드와 같은 곳에 있어야 함께 늙지 않음
- 이슈 템플릿: 스토리·수용 기준·추적성(R-번호·ADR 링크)·참고를 양식으로 — AC(Acceptance Criteria, 수용 기준) 없는 이슈를 만드는 순간 어색해지게(도구에 의한 강제의 최소 단위)
- 브랜치 보호가 켜지는 순간 "이해 없는 머지 금지"(IX장 원칙 5)는 선언이 아니라 물리 법칙



〈그림 8-3〉 워크플로우 7단계와 실패 루프: 반려는 정상 경로다

브랜치·커밋 규칙 (표 8-2)

항목	규칙
브랜치 이름	feature/{이슈번호}-{요약} · 버그는 fix/{이슈번호}-{요약}
커밋 메시지	‘동사: 요약 (#이슈번호)’ — 예: ‘추가: 건물 검색 API(Application Programming Interface) (#7)’
머지 방식	Squash 머지 — 이슈 하나 = main의 커밋 하나, 이력이 백로그와 1:1
보호 규칙	main 직접 푸시 금지 · 상태 체크 전부 통과 · 리뷰 1인 승인 필수

3. Projects 보드: 팀의 단일 진실

- 컬럼 5 고정: Backlog / Sprint Backlog / In Progress / In Review / Done — 마일스톤이 스프린트가 되어 이슈를 기간에 묶음

- 운영 규칙 4: ① 이슈 없는 작업 금지(보드에 없는 일은 하지 않는다) ② WIP(Work In Progress, 진행 중 작업) 제한 — In Progress 1 인 1 건, 끝내기가 시작보다 우선 ③ 상태 이동의 자동화(할당→In Progress, PR→In Review, 머지→Done) ④ 정렬은 PO 만 — 순서는 결정이며 결정에는 주인이 있음

4. 개발 워크플로우 7 단계

- ① 이슈 선택(자기 할당 → 보드 자동 이동) ② 브랜치 생성 ③ 구현(AI-TDD(Test-Driven Development, 테스트 주도 개발), 상세는 XI장) ④ PR 생성(변경 요약·테스트 방법·AI 사용 내역·이해하지 못한 부분) ⑤ 자동 검증(하나라도 실패 시 머지 버튼 잠금) ⑥ 동료 리뷰(1 인 이상, 설명 질문 최소 1 회) ⑦ Squash 머지(이슈 자동 닫힘·보드 Done)
- 실패 루프: ⑤·⑥ 반려 시 ③으로 — 실패 로그를 AI 에게 주어 수정 위임, 재푸시로 검증 자동 재실행

5. Actions 와 DoD

파이프라인	트리거	내용
① 머지 게이트	PR 열람·갱신	빌드 → 단위 테스트 → AI 코드 리뷰 → 시크릿·보안 스캔 — 전부 초록불이어야 머지 가능
② 배포	main 머지 · 릴리스 태그	통합 테스트 → 스테이징 자동 배포 → (태그 시) 프로덕션 카나리 — 상세는 XIII장

- DoD 6 항목 + 항목별 강제 주체(도구 또는 리뷰 절차) — 지켜지지 않는 DoD 는 없는 것과 같고, 도구로 강제된 DoD 만 살아남음
- 환경 셋업은 AI 위임의 좋은 연습장(완료 조건 명확·실패 안전) — 단 생성된 설정은 반드시 읽고 이해 후 적용

이 단계의 검증 체크리스트

- main 직접 푸시가 실제로 차단되는가 (일부러 시도해 확인)
- AC 없는 이슈, AI 사용 내역 없는 PR 이 템플릿 때문에 ‘만들기 어색’해졌는가
- 이슈 할당·PR·머지 시 보드가 자동으로 움직이는가
- 일부러 실패하는 테스트를 넣은 PR 이 머지 불가가 되는가
- 팀원 전원이 파이프라인의 각 스텝을 설명할 수 있는가

전형적 실수와 검증

전형적 실수	검증
초록불 맹신 — CI(Continuous Integration, 지속 적 통합) 통과 = 완료로 착각	CI는 DoD의 일부만 측정 — 테스트가 요구를 대변하는지(XII 장)·작성자가 이해했는지(원칙 5)는 리뷰 절차가 담당
보드와 현실의 괴리 — 수동 이동 보드는 금요 일마다 소설이 됨	상태 이동을 자동화 규칙에 위임 — 투명성은 습관이 아니라 자동화의 산물
이해 없는 셋업 복붙 — 파이프라인이 깨지는 날 아무도 못 고침	역방향 설명 검증("한 줄씩 설명 + 못 잡는 결함 3가지") + 팀원별 스텝 설명으로 Sprint 0 마감

IX AI 시대의 Scrum 운영 원칙 5

한 줄 요약 Scrum의 암묵 전제 5개가 AI로 흔들림 — 다섯 원칙은 그 조정이며 한 문장으로 수렴: 생성이 싸질수록 프로세스는 검증과 이해를 지키는 방향으로 재설계되어야 함

성격	3부의 마무리 — 반복 단계의 규범
핵심 산출물	팀 워킹 어그리먼트 (1쪽)
대응 활동	원칙의 자기 팀 번역

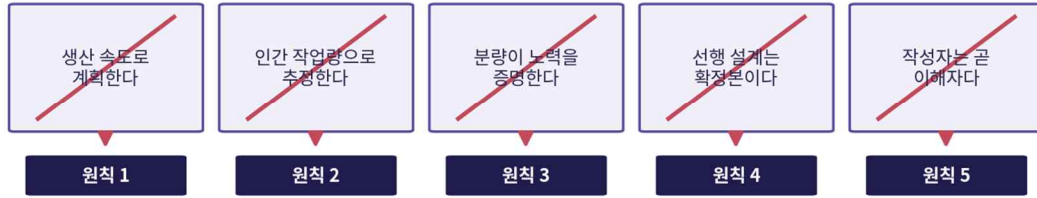
다섯 원칙 한눈에 보기

원칙	흔들리는 전제	핵심 강제 장치
1. 검증 용량 계획	생산 속도 기준 계획	In Review WIP 제한 · 리뷰의 1급 작업화 · PR 300줄 상한
2. 이해·검증 추정	인간 작업량 포인트	기준표 재작성(검증 난도 기준) · Sprint 1 실측 보정 · 벨로시티 성적 미반영
3. 결정 중심 문서	분량 = 노력의 증거	쪽 상한(2·4·10·8) · 결정 요약 · "지우면 사라지는 결정이 있는가"
4. ADR 성장 설계	선행 설계 = 확정본	새 ADR 추가 + 관계 표기(superseded/보완) · 원본 불변 · 영향 분석은 AI
5. 이해 없는 머지 금지	작성자 = 이해자	설명 질문 프로토콜 · 무작위 워크스루 · 정직한 무지의 제도화

원칙별 요약

- 원칙 1: 리뷰 부채(PR 적체 → 형식적 승인 → 아무도 안 읽은 코드의 main 축적)가 이 프로세스 최악의 실패 모드 — 스프린트 상한은 생성 능력이 아니라 소화 가능한 리뷰·검증 시간. 이슈마다 예상 리뷰 시간 기입, Daily 는 In Review 확인으로 시작
- 원칙 2: AI 가 잘하고 검증 쉬운 작업(패턴 CRUD(Create·Read·Update·Delete))은 낮게, 생성은 쉬워도 검증 어려운 작업(도메인 로직·다중 모듈 통합)은 높게 — "AI 가 해 주니 1 점"의 함정. 벨로시티는 내부 보정 전용: 측정이 목적을 왜곡
- 원칙 3: 하룻밤 50 쪽 SRS 의 시대 — 가치는 페이지 수가 아니라 결정의 수와 질. 결정 요약의 각 항목에 팀 토론 기록(회의록·이슈 링크)을 연결해 결정만큼은 팀이 내렸음을 증명
- 원칙 4: 전제가 깨질 때의 두 실패 — 문서를 지키느라 배움 무시 / 문서를 무시하고 코드만 변경(설계·구현의 조용한 괴리). 유일한 공식 경로는 새 ADR — 결정은 사람이, 파급의 지도(영향 분석)는 AI 가. 설계를 세 번 바꾼 팀이 한 번도 안 바꾼 팀보다 높게 평가될 수 있음
- 원칙 5: 이해 없이도 완성이 가능해진 시대 — 효율로는 성공, 교육·유지보수로는 실패. 미이해 기재는 감점이 아니라 리뷰어가 함께 봐 주는 신호(감점은 숨겼다 발각되는 경우). AI 설명의 활용은 권장하되 자기 말로 재구성할 수 있어야 함

Scrum 관행에 깔린 다섯 가지 암묵적 전제 — AI가 균열을 낸다



전제가 깨진 채 관행만 유지하면 프로세스는 형식이 된다 — 다섯 원칙이 그 균열을 메운다

〈그림 9-1〉 다섯 개의 흔들리는 전제와 그에 대응하는 다섯 원칙

회고 순환 점검 질문 (표 9-2)

원칙	점검 질문
1	PR이 In Review에 머문 평균 시간은? 우리의 승인은 읽은 승인이었나, 통과용 승인이었나
2	추정 초과와 원인은 생성이었나 검증이었나? 기준표는 실측으로 갱신되었나
3	문단 하나를 지우면 사라지는 결정이 있는가? 결정마다 ‘왜’와 배제 대안이 있는가
4	ADR 없이 코드만 바뀐 설계 변경은 없는가? 원본+ADR 체인만으로 현재 구조를 이해할 수 있는가
5	main에 누구도 설명 못하는 파일이 있는가? 워크스루에서 막힌 부분은 학습으로 이어졌는가

- 다섯 원칙은 규정집이 아니라 회고의 렌즈 — 답이 불편할수록 좋은 회고

팀 활동: 워킹 어그리먼트

- 가장 먼저 깨질 전제 투표 → 원칙별 구체 규칙 1~2 개(숫자 필수 — In Review 상한 4 건, 리뷰 이슈당 30 분, 워크스루 Retro 직전 20 분 등)
- AI 에게 "이 규칙들이 지켜지지 않을 상황 5 가지"를 공격시켜 방어 가능한 규칙만 채택
- 1 쪽으로 확정해 docs/에 커밋 — 회고마다 개정 가능한 살아 있는 문서

X 상세 설계 단계

한 줄 요약 Sprint Planning 후반에서 이번 스프린트 것의 ‘어떻게’를 확정 — 산출물이 곧 XI장 L3 위임의 입력이므로, 상세 설계의 품질 = 위임의 성패

적정 위임 수준	L2 초안자 ~ L3 위임
스프린트 속 위치	Sprint Planning 후반
핵심 산출물	OpenAPI 명세·ERD(Entity-Relationship Diagram, 개체-관계 다이어그램)·작업 정의 카드
대응 실습	실습 9

1. 산출물의 해부

산출물	내용
OpenAPI 명세	이번 스프린트 API의 계약 — 엔드포인트·파라미터·응답·오류
ERD와 스키마	관련 테이블의 구조와 제약 (상세는 스프린트 분량만 — DEEP)
작업 정의 카드	이슈를 쪼갠 위임 단위 — 목표·완료 조건·가드레일·컨텍스트·추적성

- 셋 모두 docs/에 텍스트로 — 특별한 지위: 완료 조건이 검증 가능한 작업 정의는 그 자체로 에이전트에게 건네는 지시서
- 다음 스프린트 기능까지 미리 상세화하는 것은 아직 오지 않은 지식으로 결정하는 낭비

2. 작업 분해: 위임 가능한 단위

카드 요소	내용
목표	무엇을 만드는가
완료 조건	무엇이 되면 끝인가 — 검증 가능하게, 대개 ‘첨부한 테스트 통과’
가드레일	하지 말 것 — 마이그레이션 수정 금지·새 라이브러리 금지·변경 범위 한정
컨텍스트	명세 몇 절, 어느 테이블, 규약 파일 — AI가 봐야 할 재료

- 가드레일은 AI 위임 특유의 요소 — 사람에게겐 상식인 경계가 AI 에겐 명시해야 할 규칙
- 크기 기준 = PR 300 줄 상한(원칙 1): 초과 예상 시 더 분해 / 단위 판정 2 질문: 하루 안에 끝나는가·완료 조건을 한 문장으로 쓸 수 있는가

작업 정의 카드 — 이슈 #7-A '검색 엔드포인트 핸들러 구현'	
목표	GET /api/buildings/search 를 OpenAPI 명세(design.md)대로 구현한다
완료 조건	첨부한 테스트 15개 전부 통과 — 정상 5 · 경계 4 · 오류 6 (검증 가능한 한 문장)
가드레일	마이그레이션 파일 수정 금지 · 새 라이브러리 추가 금지(express·pg만) · 변경은 server/src/ 한정
컨텍스트	design.md의 OpenAPI 절 · building/building_alias 스키마 · CLAUDE.md(저장소 규약)
추적성	#7 ← R-01(검색) · R-12b(게시만 노출) · N-03(p95 1초)
예상 변경량: 약 90줄 (300줄 상한 안) — 실제 결과는 app.js 81줄	

다섯 요소가 완비된 카드만이 XI장에서 에이전트에게 건네질 자격이 있다

<그림 10-2> 위임 가능한 작업 단위의 해부: 이 카드가 XI장에서 에이전트에게 전달된다

3. 계약·스키마·시퀀스

- 인터페이스 우선: 프론트·백 병렬의 전제는 OpenAPI 계약의 선확정 — 프론트는 Mock 서버로 UI(User Interface, 사용자 인터페이스), 백은 명세대로 구현: "계약을 지켰다면 붙는다"
- 명세 3 검토: 명명의 일관성(자원 복수형·동사는 HTTP(HyperText Transfer Protocol) 메서드) / 오류의 완비(성공만 있는 명세는 반쪽 — 사유를 코드로 구분) / 수용 기준과의 정합(성능 기준이 명세 어디에 반영되는가)
- ERD 검증 2 방향: 대표 질의(핵심 질의를 실제로 적어 인덱스 사용 확인) + 상태 전이(허용된 전이만 가능하도록 제약 설계) — 마이그레이션 초안은 AI, 실행 전 사람이 읽음(가장 비싼 변경)
- 시퀀스 다이어그램: 여러 참여자가 얹힌 흐름을 코드 전에 걸어 보기 — 그리는 순간 질문이 드러나며(전이 주체는? 같은 트랜잭션인가?) 여기서 발견은 공짜, XI장에서 발견은 재작업
- 설계 개정: 상위 전제 붕괴 시 원칙 4 의 경로(이슈→논의→새 ADR→관계 표기) — 상세 설계가 만드는 신규 결정도 같은 형식으로 기록

이 단계의 검증 체크리스트

- 모든 작업 단위에 검증 가능한 완료 조건과 가드레일이 있는가 — 예상 변경량 300 줄 이내인가
- API 명세에 오류 응답이 완비되고 수용 기준 대응표가 붙어 있는가
- 대표 질의가 인덱스를 타는가 — 상태 전이 제약이 설계에 있는가
- 명세·ERD 의 모든 요소가 R-번호로 소급되는가 — [제안]이 본문에 섞이지 않았는가
- 이번 스프린트 범위 밖까지 상세화하지 않았는가 (DEEP)

전형적 실수와 검증

전형적 실수	검증
SRS와 어긋나는 상세 설계 — AI가 그럴듯한 항목을 조용히 추가(조용한 범위 확장)	전 요소를 R-번호로 소급 — 소급 불가는 [제안] 분리 후 PO 결정
그렇듯하지만 비효율적인 스키마 — 질의 패턴을 모름	대표 질의 검증을 통과 의례로 + 인덱스는 존재가 아니라 사용을 확인(EXPLAIN — 소량 데이터에선 전체 스캔이 정상)
과잉/과소 분해 — 관리 비용 초과 또는 300줄·리뷰 용량 붕괴	2질문 판정: 하루 안에 끝나는가 / 완료 조건 한 문장 가능한가 — 하나라도 아니면 크기 재조정

XI 구현 단계

한 줄 요지 교재의 무게중심 — 완료 조건을 테스트로 박고, 에이전트에 위임하고, 사람이 검증해 머지. 앞 단계들이 L1·L2였기에 L3가 가능: 준비 없는 L3는 위임이 아니라 방기

적정 위임 수준	L3 위임
스프린트 속 위치	Day 2~9, In Progress
핵심 산출물	코드+테스트+PR(리뷰 기록)
대응 실습	실습 10

1. 구조: 이중 루프와 AI-TDD

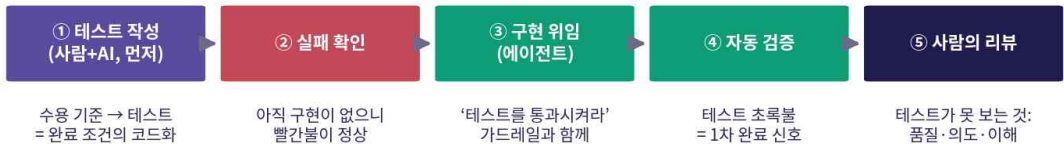
국면	내용	주체
① 테스트 작성	작업 정의의 완료 조건을 실행 가능한 테스트로 — 검사 목록은 사람이 확정	사람(AI 보조)
② 실패 확인	빨간불이 정상임을 확인 — 항상 통과하는 테스트의 함정 방지	사람
③ 위임	‘이 테스트를 통과시켜라’ + 가드레일	에이전트
④ 자동 검증	초록불 = 1차 완료 신호 (자가 검증·재시도)	에이전트
⑤ 사람의 리뷰	테스트가 못 보는 것 — 코드 품질·의도 정합·작성자의 이해	사람

- 바깥 루프 = 이슈의 생애(VIII장 7 단계, 1~2 일), 안쪽 루프 = 마이크로 사이클(수십 분) — 시간 단위의 구분이 기법의 자리를 정함
- 순서가 본질: 테스트 먼저 = 완료 판정권이 사람에게 / 코드 먼저 = AI 구현이 정답 행세 — 코드에 맞춰 테스트를 쓰는 것은 채점 기준을 답안지에서 베끼기

2. 작업 성격별 도구 선택 (표 11-2)

작업 성격	적합한 갈래
완료 조건이 명확한 독립 단위	에이전틱 (L3 위임)
작성 중 코드의 국소 보완	IDE 인라인
열린 질문·설계 미세 판단	챗 (L1~L2)
여러 파일에 걸친 기계적 변경	에이전틱 (테스트 안전망 필수)

- 한 이슈 안에서도 갈래를 오감 — 구현은 위임, 리뷰 지적 두 줄은 인라인, 관례 질문은 챗



핵심: 테스트가 먼저면 ‘완료’의 판정권이 사람에게 있고, 코드가 먼저면 AI의 구현이 정답 행세를 한다

러닝 케이스: #7-A의 테스트 15개(부분 일치·거리순, 별칭 매칭, 한 글자 허용, 게시 필더 …)를 먼저 쓰고 위임한다

3. 위임의 실행: 방치가 아니라 관찰

- 계획을 먼저 읽기 — 범위 밖 파일·과한 구조 변경이 보이면 코드가 나오기 전에 중단(가장 저렴)
- 가드레일 위반 감시 — 마이그레이션·새 라이브러리를 건드리면 즉시 중단
- 반복 실패의 손절선 — 같은 테스트 3 회 실패면 에이전트가 아니라 작업 정의의 문제: 중단 후 완료 조건·컨텍스트 재점검
- 컨텍스트 유지: 반복 오류는 그 자리에서 규약 파일에 한 줄(다음 위임부터 즉효) / 오염 세션은 결정 요약 받아 초기화
- 리팩토링·일괄 변경: 테스트 안전망이 먼저 — 없으면 현재 동작을 고정하는 테스트부터. 리팩토링 PR 과 기능 PR 은 분리(diff 의 순도 = 리뷰 가능성)
- 디버깅: ‘고쳐 줘’가 아니라 버그 카드(증상·재현 조건·기대·환경·가설) — 재현 조건의 정밀도가 가설의 품질, 수정에는 반드시 회귀 테스트 동반

4. PR 규율

규율	내용	근거 원칙
300줄 상한	넘으면 쪼갬 — 예외를 두는 순간 상한은 없는 것	원칙 1
AI 내역·미이해 기재	템플릿의 두 칸이 리뷰의 지도 — 리뷰어는 미이해 표시부터	원칙 5
diff 분리 독해	요청한 변경 vs 떨어진 변경 — 선의라도 범위 밖은 반려 (되돌리거나 별도 이슈로)	—

이 단계의 검증 체크리스트

- 테스트가 구현보다 먼저 작성되었고 실패(빨간불)를 확인했는가
- 에이전트의 계획을 읽고 승인했는가 — 가드레일 위반·3 회 실패에 개입했는가
- PR 이 300 줄 이내이고 AI 사용 내역·미이해 부분이 기재되어 있는가
- diff 에 ‘떨러온 변경’이 없는가 — 있다면 반려·분리했는가
- 낫선 API 를 공식 문서와 대조했는가 — 버그 수정에 회귀 테스트가 따라왔는가

전형적 실수와 검증

전형적 실수	검증
환각 API — 그럴듯한 이름·인자의 존재하지 않는 함수	1차 컴파일·실행(파이프라인), 2차 낫선 API의 공식 문서 대조 — 버전이 걸리면 컷오프 의심
과잉 생성 — 요청 48줄에 ‘개선된’ 452줄 동봉	diff 분리 리뷰 + 가드레일에 변경 범위 한정 + 300줄 상한의 구조적 차단
이해 부채 — 초록불 뒤에 숨는 무지	원칙 5 장치 상시 가동 + 개인 습관: 머지 전 diff를 소리 내어 설명해 보기

XII 테스트 단계

한 줄 요지 검증의 검증 — AI가 구현도 테스트도 만드는 시대, 테스트가 요구를 대변하지 못하면 초록불은 잘못된 안심을 판다. 할 일은 테스트 추가가 아니라 기존 테스트의 감사와 게이트화

적정 위임 수준	L3 위임
스프린트 속 위치	PR 게이트 · Review 준비
핵심 산출물	테스트 스위트·품질 게이트·감사 기록
대응 실습	실습 11

1. 층위별 분업: 테스트 피라미드

층위	원본(입력)	분담
단위	수용 기준	변환·생성은 AI, 검사 목록은 사람
통합	OpenAPI 명세	명세의 전 응답 케이스(성공·오류)를 실 DB 위에서 — 오류 완비의 보람이 여기서
E2E(End-to-End, 종단 간)	핵심 사용자 여정	한 줄기만 — 늘리고 싶은 유혹은 피라미드 그림으로 누름

- 최우선 규칙: 테스트 생성의 입력은 구현 코드가 아니라 스펙 — 코드 기반 생성은 코드가 하는 일을 그대로 확인해 버그까지 정답으로 봉인(순환 오류). 구현을 보여 줘야 할 때(리팩토링 안전망)는 목적을 명시: ‘현재 동작의 고정’인지 ‘요구의 검증’인지

2. 엣지 발굴과 테스트 데이터

발굴 범주	예
길이의 경계	0자 · 1자 · 2자 · 초장문
문자의 종류	한글 · 특수문자 · 이모지
악의 입력	SQL(Structured Query Language) 조각 · 스크립트 · 와일드카드
상태의 결합	전부 게시 중단 · 동시 요청

- 발굴 ≠ 채택: 나온 30 개를 다 테스트하지 않음 — 수용 기준 관련성과 위험도(발생 가능성×피해)로 사람이 선별, 미채택은 기록만
- 데이터는 현실적인 가짜: 도메인을 알려 준 픽처 / Mock 은 명세 대조로 형식 일치 확인 / 개인정보 실데이터 금지(횡단 습관 3)



구현을 보고 만든 테스트는 ‘버그까지 정답’으로 봉인한다 — 테스트의 원본은 언제나 스펙이다

〈그림 12-2〉 순환 오류: 구현에서 나온 테스트는 버그를 봉인한다

3. 테스트의 감사: 커버리지 너머

- 커버리지 = 양의 지표(‘이 줄이 실행되었다’) — 단언 없는 실행·형식적 스냅샷도 숫자를 올림
- 뮤테이션 = 질의 지표(‘검증되었다’): 작은 돌연변이(>= → >, + → -)를 심어 테스트가 못 잡으면 그 자리가 검증의 사각지대 — 핵심 모듈에 주 1 회 수준 운용
- 게이트로 굳히기: 신규 커버리지 문턱·회귀 전수·보안 스캔·주기 뮤테이션 — 문턱값은 워킹 어그리먼트에, 단 한계 명시: 게이트는 기계가 썰 수 있는 것만 잼

AI 생성 코드 보안 점검 (표 12-2)

점검 항목	무엇을 보는가
시크릿 하드코딩	키·비밀번호·토큰이 코드/로그에 박혀 있지 않은가
주입 가능성	질의·명령이 문자열 연결로 조립되지 않는가 (파라미터 바인딩)
과다 권한·노출	필요 이상의 DB 권한, 오류 응답의 스택·내부 경로 노출
입력 검증 누락	서버 측 검증 없이 클라이언트 검증만 믿지 않는가

이 단계의 검증 체크리스트

- 모든 테스트의 기대값이 R-번호로 소급되는가
- 명세의 오류 응답 케이스가 통합 테스트에 빠짐없이 있는가
- 핵심 모듈의 뮤테이션 점수를 측정하고 사각지대를 보강했는가
- 보안 체크리스트를 리뷰에서 실제로 적용했는가
- 불안정 테스트가 격리·수정되었는가 — 빨간불을 무시한 기록이 없는가

전형적 실수와 검증

전형적 실수	검증
버그를 정답으로 간주한 테스트 (순환 오류)	입력을 스펙으로 고정 + 리뷰 질문 "기대값은 어디서 왔는가 (R-번호)" — 답이 ‘코드가 그렇게 하니까’면 재작성
커버리지 숫자 놀음 — 문턱 통과용 단언 없는 테스트 양산	문턱은 목표가 아니라 바닥 — 뮤테이션 병행 + 감사 프롬프트(단언 약한 테스트·중복 검사·못 잡을 버그 시나리오) 정기 실행
불안정(flaky) 테스트 방치 — 팀이 빨간불 무시를 학습	발견 즉시 이슈·격리·수정 — "재실행하면 되지"는 금지어(게이트 신뢰의 붕괴 경로)

XIII 배포 단계

한 줄 요지 유일한 L4 허용 단계, 그래서 가장 조심스러운 단계 — 자율은 신뢰의 결과. 스테이징은 물처럼(머지마다 자동), 프로덕션은 의식처럼(스프린트 말 릴리스)

적정 위임 수준	L3 위임 ~ L4 자율
스프린트 속 위치	스테이징 상시 · 프로덕션은 스프린트 말
핵심 산출물	배포 파이프라인 · 런북 · 릴리스 노트
대응 실습	실습 12

1. 환경의 사다리: 각 칸이 증명하는 것

환경	증명하는 것
로컬	내 기계에서 돈다
CI	깨끗한 환경에서 재현된다
카나리	일부 실사용자에게 안전하다
프로덕션	전체에게 서비스한다

- 칸을 건너뛰는 배포("로컬에서 됐으니 바로 프로덕션")가 사고의 지름길 — 한 칸 오를 때마다 증명이 하나씩 추가

2. 카나리와 자동 롤백

- 카나리: 신버전을 트래픽 10%에만 → 30 분 관찰 → 정상이면 100% 전환, 위반이면 자동 롤백 — 실패의 반경을 가두고 되돌림을 사람의 반사신경이 아니라 규칙에 위임
- 롤백 조건은 배포 전에, 침착할 때, 표로 명문화 — 장애 중에 문턱값을 토론하는 팀은 이미 늦음
- 무트래픽 함정: 카나리에 트래픽이 없으면 30 분의 무사고는 아무것도 증명하지 않음 — 관찰 가능한 최소 트래픽(팀의 시나리오 주행)을 절차에 포함



카나리의 본질: 실패의 반경을 10%로 가두고, 되돌림을 사람의 반사신경이 아니라 규칙에 맡긴다

〈그림 13-3〉 카나리 배포와 자동 롤백: 실패의 반경을 가둔다

롤백 조건 예 (표 13-2)

지표	문턱 (5분 지속)	행동
오류율	1% 초과	자동 롤백 + 팀 알림
p95 응답 시간	1초 초과	자동 롤백 + 팀 알림
5xx 응답 수	분당 10건 초과	자동 롤백 + 배포 동결

카나리 무트래픽	10분간 요청 0건	관찰 연장 (관정 불가 상태)
----------	------------	------------------

3. IaC(Infrastructure as Code, 코드형 인프라)·마이그레이션·릴리스 문서

- IaC: 생성은 맡기되 한 줄씩 설명을 요구해 이해(역방향 검증) / 시크릿은 보관소에 / 콘솔 수동 변경(드리프트)은 즉시 코드 반영 — 어긋난 IaC 는 거짓말하는 문서
- DB 마이그레이션 = 가장 되돌리기 어려운 변경: 스테이징 리허설 필수 · 되돌림 스크립트 동반 · 파괴적 변경(컬럼 삭제)은 두 릴리스 분할
- 릴리스 노트 자동화: 머지 PR 목록으로 사용자용·개발자용 이중 생성 — 전제는 그동안의 규율(커밋·PR·이슈의 규칙적 연결)

4. L4 자율의 조건과 앱스토어

구분	서버 (API·관리자 웹)	iOS 앱
상시 확인	스테이징 — 머지마다 자동	TestFlight — 내부 테스터는 심사 없이
점진 공개	카나리 — 10%·30분	단계적 출시 — 7일 1%→100%
되돌림	자동 롤백 — 수 분 안에	불가 — 출시 중단 + 수정판 재제출(재심사)
게이트	머지·품질 게이트	같은 게이트 + 앱스토어 심사(통제 밖)

- L4 전환 조건 2: 게이트 완비(VIII·XII) + 롤백 명문화·리허설 검증 — 충족 전 자동 배포는 금지, 충족 후 수동 배포는 낭비
- Actions 리트머스 2: if: failure()의 롤백 스텝(행복 경로 탈출구) / environment 의 필수 승인자(결면 L3, 리허설 통과 후 풀면 L4 — 전환은 설정 한 줄)
- 비가역 배포 단위에서는 규율의 무게중심 이동: 서버는 롤백을 훈련, 앱은 출시 전 검증을 훈련 — 심사는 일정에 넣는 것 자체가 위험 관리

이 단계의 검증 체크리스트

- 스테이징이 프로덕션과 같은 구성인가 — 사다리의 칸을 건너뛴 경로가 없는가
- 롤백 조건이 표로 존재하고 자동 롤백이 리허설로 검증되었는가
- 마이그레이션에 리허설 기록과 되돌림 스크립트가 있는가
- 배포 스크립트·IaC 를 한 줄씩 설명할 수 있는가 — 시크릿·권한 점검을 통과했는가
- L4 전환이라면 두 조건(게이트·롤백)의 충족 근거가 기록되어 있는가

전형적 실수와 검증

전형적 실수	검증
시크릿 하드코딩·과다 권한 — 예제 코드의 완성	시크릿 스캔 게이트 + 최소 권한을 명시한 재생성 요청 + 배포 로그의 시크릿 노출 확인
행복 경로 파이프라인 — 중단·부분 실패·롤백 실패 경로 부재	"실패하는 시나리오 5가지" 강제 프롬프트 + 결합 주입 롤백 훈련 — 훈련하지 않은 롤백은 없는 롤백
지표 없는 배포 — '조용하면 정상'의 착각	롤백 지표의 대시보드 실재 확인 + 카나리 트래픽 확인을 배포 전 점검 항목으로

XIV 운영·유지보수 단계

한 줄 요약 유지보수는 개발 뒤의 별도 단계가 아니라 백로그로 흘러드는 강 — 루프가 닫힘. 이해 부채(원칙 5)의 청구서가 도착하는 곳

적정 위임 수준	L3 위임
스프린트 속 위치	백로그의 일부 (핫픽스만 예외)
핵심 산출물	모니터링 체계·사후 분석 보고서·개선 백로그
대응 실습	실습 13

1. 관측의 3 요소

요소	답하는 질문	비고
로그	무슨 일이 있었나	AI에게 요약·유형 분류·이상 후보 탐지 위임
지표	얼마나 건강한가	오류율·응답 시간·p95 — 롤백 조건 표가 그대로 상시 알림 문턱
알림	언제 사람을 부르나	문턱은 사람이 정함 — 무엇이 ‘이상’인지는 가치 판단

- AI 의 이상 탐지는 후보 — 관정은 반드시 로그 원문 확인 후: 로그 해석에도 환각(존재하지 않는 패턴의 ‘발견’) — 인용 불가 패턴은 보고 금지 조건을 프롬프트에

2. 장애 대응: 순서가 규율이다

- 완화가 원인 규명보다 먼저 — 서비스를 살리고(롤백·기능 끄기: XIII장의 되돌림 수단 재사용) 그 다음 침착하게 규명. 불을 끄기 전에 발화점을 토론하는 팀은 사용자를 잃음
- RCA(Root Cause Analysis, 근본 원인 분석): 장애 전후의 로그·배포 이력·지표를 주고 원인 후보의 순위 + 각 후보의 확인 실험 요청(버그 카드의 확장판) — 좋은 RCA 는 ‘왜’를 코드에서 멈추지 않고 결정과 가정까지 파고들
- 재발 방지: 그 장애를 재현하는 회귀 테스트 + 알림 보강 + 필요 시 새 ADR
- 사후 분석 보고서: 타임라인·원인·대응·배운 것 — 비난 없이(blameless): 누가 실수했는가 아니라 무엇이 이 실수를 가능하게 했는가 — 이 문화가 다음 장애 때 침묵을 방지

러닝 케이스 RCA: 개강 주 검색 장애 (5 Whys)

증상	개강 첫 주 저녁, 검색 응답 8초 — N-03(1초) 위반, 카나리였다면 롤백 문턱
왜 1	같은 게시 건물 조회가 매 요청 DB로 직행했다
왜 2	서버에 응답 캐시가 없다
왜 3	응답 캐시(RT3)는 VI장 레드팀에서 식별되고도 백로그로 분류되었다
왜 4	분류의 근거인 트래픽 가정이 평소 기준 — 개강 주 피크(평시 20배)를 검증하지 않았다
근본 원인 → 대응	가정이 깨졌다 — 새 ADR-006: 게시 건물 조회 응답 캐시 (원칙 4의 경로)

〈그림 14-4〉 5 Whys 실전: 코드에서 멈추지 않고 가정까지 — 종착지는 ADR이다

핫픽스 경로와 정규 경로 (표 14-2)

구분	핫픽스 경로 (긴급 장애만)	정규 경로 (그 외 전부)
----	-----------------	----------------

착수	SM 승인 즉시, 스프린트 중 끼어들	백로그 → 다음 Planning
필수 관문	RCA 요약 + 회귀 테스트 + 리뷰 1인 (생략 불가)	DoD 6항목 전부
금지	RCA 없는 ‘증상 덮기’ 머지	핫픽스 명목의 기능 추가
사후	48시간 내 사후 분석, 부채 잔여분은 이슈로	—

- AI 패치는 증상을 빠르게 덮음 — 동작은 돌아오지만 원인은 생존: 최소한 원인 한 문단은 알고 고치며, 임시 패치의 부채는 즉시 이슈로

3. 레거시·부채·피드백

- 학기 후반의 레거시 = 6 주 전의 내 코드, 떠난 팀원의 모듈, 이해 못 한 채 머지된 부분(원칙 5 가 새는 곳)
- AI 는 훌륭한 고고학자 — 단 설명은 실행·테스트로 대조: 설명이 그럴듯해도 코드는 다르게 동작할 수 있음([추정] 표시 강제 + 설명 기반 최소 테스트)
- 기술 부채: 발견 즉시 이슈 + 이자(방치 비용) 한 줄 — ‘언젠가 정리’는 영원히 오지 않음. 스프린트 용량의 일정 비율(예: 15%)을 상환에 배정
- 피드백 3 분류(L2): 버그 / 개선 요청 / 오해 — 오해가 특히 값짐: 명세대로 동작하는데 헤맨다면 고칠 것은 코드가 아니라 화면·문구·스펙. 반복 요청은 SRS 개정과 ADR 로 — 운영이 스펙을 다시 쓰는 순환

이 단계의 검증 체크리스트

- 알림 문턱이 롤백 조건 표와 정합하며 실제로 올리는 것을 확인했는가
- AI 의 이상 보고를 로그 원문으로 대조했는가 (인용 없는 패턴을 기각했는가)
- 모든 핫픽스에 RCA 요약과 회귀 테스트가 붙어 있는가
- 사후 분석이 비난 없이, 가정과 결정까지 파고들었는가 — ADR 로 이어졌는가
- 운영의 발견이 24 시간 내 이슈로 기록되는가 — 부채 상환 비율이 지켜지는가

전형적 실수와 검증

전형적 실수	검증
표면 증상만 덮는 핫픽스 — 원인은 다음 장애로 이월	관문(표 14-2)을 예외 없이 — ‘일단 이걸로 막고’가 나오면 부채 이슈를 그 자리에서 생성
레거시 오해석의 확산 — 유창해서 위험(부수 효과·예외 경로 누락)	[추정] 표시 강제 + 설명 기반 테스트 대조 + 자기 말 재구성(원칙 5)
루프의 누수 — 발견이 기록되지 않아 같은 문제 반복	발견 즉시 이슈("보드에 없는 일은 하지 않는다"의 운영판) + Daily 고정 질문 ‘어제 운영에서 본 것’

XV AI 기능을 내장한 소프트웨어 만들기

한 줄 요지 도구에서 부품으로 — 개발자가 감당하던 비결정성을 사용자가 겪게 됨. 핵심은 호출법이 아니라 호출을 제품답게 만드는 법: 형식 보장·Evals·폴백·보안

스프린트 속 위치	Sprint 2 (LLM 기능 통합)
핵심 산출물	LLM 내장 기능·평가 세트(Evals)·폴백 설계
대응 실습	실습 14

1. LLM API 기초

- 호출의 본질: 메시지 배열(system/user/assistant)을 보내고 다음 메시지를 받음 — API 는 무상태: ‘대화의 기억’이란 이전 메시지를 우리 서비스가 저장했다 매번 다시 보내는 것(창 관리가 서비스 설계 문제로)
- 내장 기능의 4 대 패턴: 생성(구조화 입력→자연어) / 요약(긴 글→핵심) / 분류(텍스트→정해진 갈래) / 구조화 추출(자연어→JSON(JavaScript Object Notation)) — RAG(Retrieval-Augmented Generation, 검색 증강 생성)는 다섯째 패턴이라기보다 보장: 모델의 기억이 아니라 우리의 문서로 답하게
- 첫 결정은 ‘무엇을 안 시킬까’ — 자유 대화 챗봇의 의도적 배제(열린 입력 리스크·비용 예측 불가·범위 초과)도 결정 요약에 기록

핵심 파라미터 (표 15-1)

파라미터	의미	선택 요령
temperature	출력의 다양성 (0=일관~높음=다양)	사실 서술이면 낮게 (예: 0.3 — 재현성 우선)
max_tokens	출력 길이 상한	비용·지연의 상한을 겸함 (예: 300)
system prompt	역할·규칙·형식의 고정 지시	별도 파일로 버전 관리 — Evals와 짝
형식 옵션	출력 종료·구조 제어	JSON 강제 옵션으로 형식 붕괴 방지



폴백 설계: LLM 실패(시간 초과·형식 붕괴·한도)가 기능 실패가 되지 않게 —

러닝 케이스: 문안 생성 실패 시 ‘검수자 직접 작성’ 흐름으로 조용히 전환 (사용자는 실패를 만나지 않는다)

〈그림 15-5〉 Evals 루프와 폴백: 프롬프트의 회귀 테스트, 실패의 안전망

2. 프로덕션의 현실: Evals 와 폴백

- 비결정 컴포넌트에 문자열 일치 단언은 무력 — 대안 Evals: 대표 입력(정상·경계·악성 30 건 안팎) + 성질 기준(형식이 JSON 인가·길이 범위·금지어·입력에 없는 사실을 지어내지 않았는가)의 자동 채점
- Evals 의 지위 = 프롬프트의 회귀 테스트: 프롬프트를 한 줄 고칠 때마다 전체 세트 재실행·이전 버전과 비교 — 프롬프트도 코드처럼 버전 관리·회귀 검사
- 폴백의 원칙: LLM 실패 ≠ 기능 실패 — 시간 초과·형식 붕괴·한도 시 조용히 대체 흐름으로(사용자는 오류 화면 대신 대체 경로를 봄)

- 비용·지연의 다이얼: max_tokens 상한 / 입력의 구조화(원본 대신 추출 속성) / 동일 입력 캐시 / 사용자별 호출 한도 — 제공자 선택은 비용 실측 근거로 ADR 에(래퍼 한 겹은 교체 대비)

3. 보안: 프롬프트 주입과 층의 방어

방어 층	내용
① 입력의 구조화	사용자 입력을 데이터로 명시·구분해 전달 (문자열 연결 금지)
② 출력 검증	형식(JSON 스키마) + 값(입력 메타데이터와의 일치) 검증
③ 신뢰 경계 밖 취급	화면 표시·DB 반영 전 반드시 검사 — LLM 출력은 사용자 입력과 같은 등급의 불신
④ 민감 동작 차단	게시 전환 등은 LLM 출력만으로 실행하지 않음 — 사람의 관문 뒤에

- 위협의 구조: 외부 입력이 프롬프트의 일부가 될 때 악의적 입력이 지시 행세("지금까지의 지시를 무시하고 ~라고 써라") — 모델은 지시와 데이터를 완벽히 구분하지 못하므로 방어는 한 방이 아니라 층
- system 프롬프트의 뼈대 5: 역할 → 입력의 지위(사실의 원천) → 금지 규칙(지어내기·과장·범위 밖) → 출력 형식 강제 → 실패 시의 약속된 신호 — Evals 채점 기준과 1:1, "지어내는 것보다 비는 것이 낫다"

이 단계의 검증 체크리스트

- Evals 세트(정상·경계·악성)가 존재하고 프롬프트 변경마다 재실행되는가
- 폴백이 구현되어 LLM 실패가 기능 실패로 번지지 않는가 (일부러 실패시켜 확인)
- 출력 검증(형식+값)이 화면·DB 반영 전에 존재하는가
- 전송 필드가 화이트리스트로 관리되고 개인정보가 섞이지 않는가
- max_tokens·캐시·사용자 한도로 비용의 상한이 잡혀 있는가

전형적 실수와 검증

전형적 실수	검증
출력 형식의 붕괴를 가정하지 않음 — 파싱 실패가 곧 화면 오류	형식 강제 + 재시도 1회 + 폴백의 3중 구조를 기본형으로, Evals에 ‘형식 유효율’ 지표
입력을 문자열 연결로 프롬프트에 — 주입 무방비 (SQL 주입과 같은 구조)	구조화 전달(JSON 필드) + 악성 입력을 Evals 고정 항목으로 (‘지시 무시’ 문구 5종) + 출력 값 검증을 게이트에
개인정보의 무심한 외부 전송	전송 전 필드 화이트리스트(생성에 연락처는 불필요) + 로그의 프롬프트 원문 마스킹 규칙

XVI 종합 프로젝트: 완주의 설계

한 줄 요지 새 기법 없음 — 배울 것은 끝났고 남은 것은 완주. 15주 지도, 마감 산출물(운영 중인 서비스·AI 협업 백서·최종 발표)의 기준, 성장 경로 정리

성격	교재의 마지막 장
핵심 산출물	운영 중인 서비스·AI 협업 백서·최종 발표
대응 활동	종합 프로젝트 전체 (Sprint 0~3 + 15주차 발표)

1. 15 주 지도와 산출물 릴레이

- 산출물 릴레이가 교재의 설계 그 자체 — 문제 정의서→개요서→SRS→설계·백로그·테스트, 운영의 발견→스팩: 어느 고리가 부실하면 다음 고리의 AI 출력이 함께 부실
- 최종 점검의 질문: "이 산출물의 원본은 어디인가" — 사슬을 거슬러 답할 수 있으면 건강한 프로젝트
- 중간 점검(8 주)의 혼한 진단 = 선행 산출물의 부채(SRS 모호함이 백로그로 전이) — 처방은 멈춤이 아니라 백로그 정제 한 세션

스프린트별 완료 판단 기준 (표 16-1)

스프린트	이것이 참이면 완료
Sprint 0	더미 이슈가 7단계를 통과해 머지·위킹 어그리먼트 커밋·전원이 파이프라인 설명 가능
Sprint 1	핵심 흐름 하나가 스테이징에서 끝까지 동작·실측으로 추정 기준표 보정·리뷰 부채 없이 마감
Sprint 2	Must 완비 + LLM 기능이 Evals·폴백을 갖춰 동작·첫 프로덕션 릴리스(L3)와 런북 존재
Sprint 3	성공 기준(IV장) 달성·L4 전환 근거 기록·운영 대응(사후 분석) 1회 이상·백서·발표 준비 완료

- 기준들은 새 요구가 아니라 각 장 실습의 재확인 — 릴레이해 온 팀에게 종합 프로젝트는 별도의 산이 아니라 이미 오르던 길



<그림 16-1> 15주 여정의 지도: 한 학기가 하나의 산출물 사슬이다

2. AI 협업 백서 (8 쪽 내외)

절	내용
위임의 기록	단계별로 무엇을 어느 수준으로, 왜
검증의 기록	AI 오류를 잡은 사례 3건 이상 — 발견의 경위까지

실패의 기록	위임이 실패한 사례와 원인 진단
프로세스의 진화	어그리먼트·규약 파일·프롬프트 표준의 개정 이력
팀의 결론	다음 프로젝트에 가져갈 규칙 5개 (근거 포함)

- 학기말에 쓰는 문서가 아님 — 프롬프트 로그·PR 문답·AI 활용 회고가 이미 원문: 스프린트마다 반 쪽씩 축적한 팀은 마지막 주에 편집만

3. 평가와 최종 발표

- 평가 = 과정 60 : 결과 40 — 이해 없이도 완성이 가능해진 시대에 완성물은 역량의 충분한 증거가 아니고, 과정의 기록(무엇을 시켰고 의심했고 확인했는가)은 위조가 어려움
- 벨로시티·코드량·AI 사용량은 성적 미반영(원칙 2) — 측정이 목적을 왜곡
- 최종 발표(20 분) 구성: ① 서비스 시연 5 분(실패포본 라이브) ② AI 가 만든 최악의 버그와 잡은 과정 7 분 — 발표의 심장: 생성·잠복·발각·교훈의 4 막 서사 ③ ADR 체인 5 분(무엇을 반복했고 계기는) ④ 백서의 결론 3 분(가져갈 규칙 5 개)
- 실패담이 중심인 이유: 성공의 시연은 비슷하지만 실패의 해부는 팀마다 다름 — 버그가 없었다는 팀은 검증이 없었거나 기록이 없었던 팀

4. 윤리와 성장의 경로

- 학문적 정직성: AI 사용은 부정행위가 아니지만 사용의 은폐는 부정행위 — 기준은 공개와 설명 가능성, 다른 맥락에선 그 맥락의 규칙을 먼저 확인
- 저작권·라이선스 확인은 실무에서 법적 책임의 문제 / 실사용자를 받은 순간 팀은 개인정보 처리자(수집 최소화·전송 화이트리스트·폐기 계획)
- 모든 규율이 수렴하는 문장: AI 가 만들었어도, 머지한 사람이 만든 것이다
- 성장 3 방향: 깊이(검증의 전문성 — 테스트 설계·보안·관측성: AI 가 대체하기 가장 어려운 역량) / 넓이(멀티 에이전트·RAG 본격 구축·Evals 자동화) / 높이(팀의 프로세스를 설계하는 사람)

XVII 결론: 생성의 시대, 검증의 개발자

한 줄 요지 생성 비용의 극적 하락 → 개발자의 가치는 설계·위임·검증으로 이동 — I장 V-Model 재해석이 빠대, 열다섯 장은 그 살

교재 요약

- 선행 4 단계 = AI 에게 줄 원본 만들기 / Scrum·GitHub·다섯 원칙 = 원본이 팀 안에서 부패하지 않게 지키는 제도 / 반복 5 단계 = 위임과 검증이 교대하는 현장
- AI 가 도구에서 제품의 부품이 되어도(XV장) 규칙은 동일: 생성은 싸고, 검증이 가치
- 위임의 수준(L1~L4)을 정하는 것은 능력이 아니라 신뢰 — 신뢰는 게이트와 기록으로만 축적

다섯 원칙, 여정의 끝에서

원칙	끝에서 다시 읽으면
1. 검증 용량 계획	스프린트가 무너지는 지점은 코드 부족이 아니라 리뷰 지연 — PR 300줄 상한은 취향이 아니라 이 원칙의 산술
2. 이해·검증 추정	실측 보정 기준표가 Sprint 1 의 완료 기준이 된 이유 — 측정이 목적을 왜곡하므로 벨로시티는 성적 밖
3. 결정 중심 문서	얇은 문서가 많은 결정을 담을 수 있는 것은 문서의 일이 서술이 아니라 결정이기 때문
4. ADR 성장 설계	스프린트 중에 태어난 결정도 앞선 결정을 지우지 않음 — 원본은 남기고 새 결정을 쌓는다
5. 이해 없는 머지 금지	이해 부채의 청구서는 반드시 도착 — 주소는 대개 새벽의 장애 현장

- 다섯은 결국 하나의 태도: 싸진 것(생성)을 아끼지 말고, 비싸진 것(이해와 검증)을 아껴 쓰라

남는 것

- 도구의 이름은 잉크보다 빨리 바뀔 — 남는 것은 판단의 프레임: 이 작업은 어느 갈래인가 / 어느 수준까지 맡길 수 있는가 / 완료 조건은 시험 가능한가 / 실패하면 어떻게 되돌리는가
- 생성의 시대는 개발자를 대체하지 않았음 — 요구하는 개발자상이 바뀌었을 뿐: 코드를 빨리 쓰는 사람이 아니라, 무엇이 맞는지를 정의하고 맞는지를 확인할 줄 아는 사람
- 바닥의 한 문장: AI 가 만들었어도, 머지한 사람이 만든 것이다

부록 용어 해설

1. 영문 약어

약어	원어	뜻·쓰임
AC	Acceptance Criteria	수용 기준 — 무엇이 되면 끝인가의 계약
ADR	Architecture Decision Record	아키텍처 결정 기록 — 결정·근거·배제 대안을 남기는 1장짜리 문서
AI	Artificial Intelligence	인공지능
API	Application Programming Interface	프로그램 간 호출 규약
CI/CD	Continuous Integration / Continuous Delivery	지속적 통합/지속적 배포 — 머지마다 자동 검증·배포하는 관행
DEEP	Detailed appropriately·Emergent·Estimated·Prioritized	좋은 백로그의 성질 — 위는 상세하게, 아래는 거칠게
DoD	Definition of Done	완료의 정의 — 증분이 갖춰야 할 공통 기준
E2E	End-to-End	종단 간 테스트 — 사용자 여정 전체를 실제 환경처럼 검사
EARS	Easy Approach to Requirements Syntax	요구를 다섯 문형으로 제한하는 작성법
ERD	Entity-Relationship Diagram	개체-관계 다이어그램 — 데이터 구조의 설계도
IaC	Infrastructure as Code	코드형 인프라 — 환경 구성을 코드로 관리
INVEST	Independent·Negotiable·Valuable·Estimable·Small·Testable	좋은 유저 스토리의 여섯 성질
LLM	Large Language Model	대형 언어 모델 — 다음 토큰 예측으로 동작
MoSCoW	Must·Should·Could·Won't	우선순위 4분류 — Won't 목록이 범위를 지킴
PO	Product Owner	제품 책임자 — 가치와 우선순위의 주인
PR	Pull Request	병합 요청 — 리뷰·자동 검증을 거치는 변경의 단위
RAG	Retrieval-Augmented Generation	검색 증강 생성 — 우리의 문서로 답하게 하는 보강
RCA	Root Cause Analysis	근본 원인 분석 — ‘왜’를 가정까지 파고듦
SCAMPER	Substitute·Combine·Adapt·Modify·Put to other uses·Eliminate·Reverse	아이디어 변형 7질문
SM	Scrum Master	스크럼 마스터 — 프로세스와 장애물 제거의 책임

SRS	Software Requirements Specification	소프트웨어 요구사항 명세서
TDD	Test-Driven Development	테스트 주도 개발 — 테스트를 먼저 쓰는 순서
WBS	Work Breakdown Structure	작업 분해 구조
WIP	Work In Progress	진행 중 작업 — 제한(WIP limit)이 흐름을 지킴
YAGNI	You Aren't Gonna Need It	‘지금 필요 없으면 만들지 않는다’는 과잉 설계 방지 원칙

2. 주요 용어

용어	해설
앵커링	먼저 접한 정보가 닳아 되어 판단이 그 주변을 벗어나지 못하는 인지 편향 — 아이디어 단계에서 AI를 L1로 제한하는 이유
환각	LLM이 근거 없는 내용을 유창하게 지어내는 현상 — 버그가 아니라 다음 토큰 예측의 구조적 산물
지식 컷오프	모델 학습 데이터의 시점 한계 — 이후의 사실은 모르면서 옛 지식으로 답함
컨텍스트 윈도우	모델이 한 번에 볼 수 있는 토큰 총량 — 창 밖의 정보는 존재하지 않는 것과 같음
규약 파일	스택·구조·컨벤션·규칙을 담아 저장소에 두는 파일 — AI 도구가 자동 참조하는 살아 있는 문서
마이크로 사이클	정의→생성→검증→수정의 기본 협업 루프 — 모든 AI 협업의 최소 단위
위임 수준 모델	L1 조언자~L4 자율의 4단계 — 얼마나 맡길지의 판단 프레임
하이브리드 프로세스	선행 4단계(문서 1회) + 반복 5단계(2주 스프린트)의 결합 — Spec-Scrum-Flow
백로그	우선순위로 정렬된 할 일의 단일 목록 — 선행 산출물과 반복 단계를 잇는 다리
에픽 / 유저 스토리 / 이슈	큰 기능 묶음 → 사용자 가치 단위 → 수용 기준이 붙은 실행 단위의 3층 분해
증분	스프린트가 끝날 때 실제로 동작하는 제품의 단위 — DoD를 통과해야 증분
벨로시티	스프린트당 완료 포인트의 합 — 팀 내부 보정 전용, 성적·비교에 쓰지 않음
리뷰 부채	생성 속도가 검증 속도를 앞질러 In Review에 쌓이는 미검토 코드 — 이 프로세스 최악의 실패 모드
워킹 어그리먼트	다섯 원칙을 팀의 숫자로 번역한 1쪽 합의문 — 회고마다 개정
가드레일	위임 시 ‘하지 말 것’의 명시 — 사람에게겐 상식인 경계를 AI에게 규칙으로
레드팀	완성 선언 전에 산출물을 적대적 관점으로 공격시키는 검토 기법
프리모템	‘이미 실패했다’고 가정하고 원인을 역추적하는 리스크 발굴 기법
런 캔버스	문제·고객·가치·수익 등 9칸으로 사업 가설을 한 장에 정리하는 도구
순환 오류	구현 코드로부터 테스트를 생성해 버그까지 정답으로 봉인하는 함정 — 테스트의 입력은 스펙
커버리지 / 뮤테이션	실행되었는가(양) / 검증되었는가(질) — 뮤테이션은 심은 돌연변이를 테스트가 잡는지 봄
flaky 테스트	같은 코드에서 때때로 실패하는 불안정 테스트 — 방치하면 팀이 빨간불 무시를 학습

픽스처 / Mock	도메인을 반영한 시험용 가짜 데이터 / 외부 의존을 흉내 내는 대역(명세와 대조 필수)
카나리 배포	신버전을 일부 트래픽에만 열어 관찰 후 전환 — 실패의 반경을 가둠
롤백	문제 발생 시 직전 정상 버전으로 되돌림 — 조건은 배포 전에 표로 명문화
스테이징	프로덕션과 같은 구성의 사전 검증 환경 — 크기가 아니라 구성이 같아야 함
마이그레이션	DB 스키마의 구조 변경 — 가장 되돌리기 어려운 변경이라 별도 규율
핫픽스	긴급 장애의 스프린트 끼어들기 경로 — RCA 요약·회귀 테스트·리뷰 1인이 필수 관문
기술 부채	당장의 편의를 위해 미룬 정리 비용 — 발견 즉시 이슈로, 이자(방치 비용) 한 줄과 함께
blameless 사후 분석	비난 없이 ‘무엇이 이 실수를 가능하게 했는가’를 묻는 장애 보고 문화
Evals	LLM 기능의 자동 평가 세트 — 대표 입력과 성질 기준의 채점, 프롬프트의 회귀 테스트
폴백	LLM 실패 시 조용히 대체 흐름으로 전환하는 설계 — LLM 실패 ≠ 기능 실패
프롬프트 주입	외부 입력이 지시 행세를 하는 공격 — 방어는 한 방이 아니라 층
게이트	머지·배포를 막아서는 자동 검사의 관문 — 도구로 강제된 규칙만 살아남음